

INTERNSHIP REPORT · 202

# RISC-V 嵌入 全栈开发实习

丁冯德 · 2026 年 3 月 -

# 项目全景图

4 个月 · 7 个仓库 · 板子验证、文档网站、互动 Demo、课程研发与教材协作



## LicheePi 4A 开发板测试

[support-matrix + ruyisdk-dev-archive](#)

在 RISC-V 开发板上跑通系统、跑神经网络程序。先用电脑模拟，再用真实板子验证，写了完整操作文档。

3月

2 个 PR

2 份报告



## Board Docs Frontend

[DuoQilai/board-docs-frontend](#)

RISC-V 开发板文档网站。从零搭站，加了搜索和编辑器插件；6 月又做了每天自动更新内容和访问统计。

3月-6月

30+ commits

持续迭代



## OpenSiliconBet

[EnzoDing-rgb/OpenSiliconBet](#)

中科院公众科学日现场互动 Demo。论坛辩论形式，AI 角色对话，观众语音提问，现场音箱回答。

4月-5月

v0.1 发布

6 角色 AI

以下按项目逐一展开，**非按月**

# LicheePi 4A · 开发板测试与程序移植

分三步验证程序能跑：普通电脑 → RISC-V 模拟器 → 真实开发板。每一步都写了操作文档。

## Tinn：一个 200 行的神经网络程序

用 C 语言写的迷你神经网络，不依赖任何第三方库。用 16×16 像素手写数字图片来训练和识别，能输出训练进度和识别结果。适合拿来验证开发板的计算能力。

## 用 RuyiSDK 工具链来编译程序

用 Ruyi 包管理器装编译工具和模拟器，一行命令就能把同一个 C 程序编译成 RISC-V 开发板能跑的版本，不用手动配环境。

## 提交的成果

给 support-matrix 仓库提了测试报告 PR · 写了完整的移植操作文档（每一步的命令和运行结果都有）· 给归档仓库提了备份 PR

## 三步验证

①

### x86\_64 Linux

普通电脑编译运行  
确认程序本身没问题  
训练误差持续下降  
识别结果正确

↓ 通过

RuyiSDK 0.46.0

gnu-plct 工具链

QEMU 8.2+

LicheePi 4A

PR: DuoQilai/su

# Board Docs Frontend ① 网站搭建

给 RISC-V 开发板做一个文档展示网站，从零开始，改了两轮。

## 内容来源

自动读取文档仓库里的说明文件和示例代码，提取标题、描述、状态等信息，生成网页。

## 浏览方式

按芯片厂商分组浏览，点进去看具体开发板，再点进去看示例代码。左边有目录，右边看内容。

## 主要页面

首页：所有开发板卡片 + 搜索筛选。详情页：开发板属性 + 代码示例列表。示例页：说明文档 + 代码高亮。

## 第二轮改进

去掉重复标题 · 修复 README 显示问题 · 加了返回首页的入口 · 界面细节打磨。

### 技术架构

Astro

React 18

TypeScript

T

### 数据流

test-doc  
子模块

→

Markdown  
解析

→

读  
标题

### 交付

仓库 [github.com/DuoQilai/board-docs-front](https://github.com/DuoQilai/board-docs-front)

# Board Docs Frontend ② 插件集成与改进

对接 VS Code 和 Eclipse 插件团队，让网站能在编辑器里用。改了 20 多次，做了两轮。

## 在编辑器里用这个网站

网站能自动识别自己是在浏览器还是编辑器里打开的。在 VS Code 里会多出一个按钮，点了就能把当前文档发给编辑器做后续处理。

按钮显隐

消息通信

## 界面品牌化

换上了 RuyiSDK 的品牌色和 logo，统一了 UI 部分。

品牌统一

## 文档来源切换

把背后的文档仓库从不规范的临时位置迁移到了 ruyisdk 组织下。隐藏了不需要展示的模板目录。

仓库迁移

目录整理

## 配合插件团队

去学了 VS Code 和 Eclipse 的插件开发，配合他们的命名规范。

跨团队配合

从零搭建 → 插件集成 → CI 自动化，三个月持续做

# Board Docs Frontend ③ 自动化与运维

让网站自动更新内容、自动检查访问情况，减少人工维护。

## 每天自动更新内容

设了一个定时任务，每天自动拉取最新文档，如果有更新就自动提交改动。不需要手动去刷新。

## 修构建工具的问题

自动构建流程里包管理器版本不对，修了配置方式。同步文档的时候有个重置逻辑也有问题，一并修了。

## 示例分类更清楚

给开发板示例加了类型标签，比如基础、外设、网络、AI、图形界面等，网站上按类型筛选更方便。

## 加了访问统计

网站接入了访问量统计，能看大概有多少人来过。相关说明文档也整理进 docs 文件夹。

每天检查文档有没有更新

10

6月 Commits

# OpenSiliconBet · RISC-V 三国杀

github.com/EnzoDing-rgb/OpenSiliconBet · 中科院公众科学日现场互动演示

## 这是个什么

一个现场互动程序：模拟论坛辩论。观众能看到 RISC-V、x86、ARM 三个阵营的 AI 角色轮流发言，有主持人控场，还有嘉宾环节和观众提问。面向普通观众，强调好玩、能互动。

## 观众怎么玩

观众对着麦克风问问题 → 电脑识别语音转成文字 → AI 当场生成回答 → 音箱把回答念出来。整个过程观众只动嘴，不用碰键盘鼠标。

## 现场布置

笔记本接扩展坞，连了外接显示屏、麦克风、音箱。活动前一个个确认设备都能正常工作，音量合适，没有延迟。

## 技术栈

- FastAPI
- Uvicorn
- React 18
- TypeScript
- Cloudflare Tunnel

## AI 角色体系

- RISC-V 辩手
- x86 辩手
- ARM 辩手
- 主持人

## 开发过程 · 约一个月（4月中旬 - 5.17）

- 4月中下旬  
方案设计  
确定互动形式  
辩论 Demo 原型  
组会演示验证
- 5月1-10日  
AI 模型训练  
模型微调  
模型部署
- 5月11-17日  
现场部署  
现场调试  
现场演示

# 《RISC-V Linux 嵌入式实践》课程草稿

[github.com/EnzoDing-rgb/riscv-embedded-course](https://github.com/EnzoDing-rgb/riscv-embedded-course) · 我自己先写的一版，用来试水定方向

## 这门课讲什么

- 在 RISC-V Linux 开发板上，用 RuyiSDK 和 C 语言做嵌入式小项目
- 有术语表、章节安排、要买什么硬件、三个递进的动手项目
- 全程用一块板子完成，不搞太复杂的环境切换

## 怎么发布出来

- 大纲做成网页
- 打开链接就能看到
- README 也
- 填了课程进度

## 6 月主要推进

- 用大白话把大纲重写了一遍
- 第三个毕业项目改成「语音关键词识别 + 控 LED 灯」
- 加了智能环境监测盒作为综合项目说明
- 精简物料清单，把第 1 到第 7 章内容对齐

## 和正式教材

- 这版是我独立
- 方向确定后，

10 章 3 个项目 网页在线 目录

课程结构

练手到毕业

GitHub Pages



# RISC-V Linux 正式教材仓库

[github.com/DuoQilai/ruyi-riscv-linux-book](https://github.com/DuoQilai/ruyi-riscv-linux-book) · 老师主导，我参与写教案和整理仓库

## 和自研草稿的关系

我先在自己仓库里写出一版课程大纲做试水。老师认可后，进入这个正式教材仓协作：把结构理顺，把前两章真正写成能上课用的讲义和实验。

## 重整仓库

- 按章、按节分文件夹，每节一份讲义 + 一份实验说明
- 每章配一份整章幻灯片
- 单独放板子参考资料和历史旧稿

## 已写完的内容

- 第 1 章 开发
- 第 2 章 工具
- 附了 hello 和
- 讲义和实验

## 板卡参考

整理了 Liche

自研草稿 (riscv-embedded-course) → 试水定方向 · 正式教材 (ruiyi-riscv-lin

# 做过的事 & 涉及的

4 个月 · 7 个仓库 · 从板子验证到网站、互动 Demo、课程

## 开发板测试

[support-matrix](#) · [ruiysdk-dev-archive](#)

板子上跑通系统和示例程序 · 2 个 PR · 2 份报告

## Board Docs Frontend

[DuoQilai/board-docs-frontend](#)

开发板文档网站 · 插件集成 · 6 月加了自动更新和访问统

## 自研课程大纲

[EnzoDing-rgb/riscv-embedded-course](#)

自己先写的一版课 · 网页在线发布

## 正式教材

[DuoQilai/ruyi-riscv-linux-book](#)

和老师协作 · 重整仓库 · 写完前两章教案

## 网站与前端

用 Astro、React 做文档站；用 Vite、React 做互动 Demo 页面；部署在 Cloudflare Pages。

## 后端与 AI

用 FastAPI 跑辩论流程；接大模型对话和语音播报；现场用麦

板子能跑 → 文档能看 → Demo 能玩 → 课  
从底层验证到内容产出，一条线走下来